COMP1521

TUTORIAL 5

INTEGERS > BITWISE OPERATIONS> USING BITWISE

NOTE:

- Remember assignment 1 is due Friday!
- Week 5 Lab is due Week 7
- Remember there is a weekly test released over flex week

INTEGERS

Numbers

- A number at its simplest form is just something to represent a value, this means we can have many ways to represent this, specifically many different bases of numbers
  - Consider the number 42

Type	Value	Prefix	Base	Expanded Form
Decimal	42	No prefix	10	$4 * 10^1 + 2 * 10^0$
Hexadecimal	0x2a	0x	16	$2 * 16^1 + 10 * 16^0$
Octal	052	0	8	$5 * 8^1 + 2 * 8^0$
Binary	0b101010	0b	2	$1 * 2^5 + 0 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0$

The reality is we can represent values in many different bases, and all have their specific uses

Why do we have these representations of numbers

- When we usually work with numbers, we'll typically write them in decimal in our programs, but our computers do not as core computer logic at the lowest level is just 1's and 0's
- binary is fundamental to all modern computers.
  - Consider when we load a variable in C, we have int x = 42
    - But when our computer loads it into ram it would be:
      00000000 00000000 00000000 00101010

- Side Note:
  - Hexadecimal you will find often used for memory addresses or colour codes (#13B0CF)
  - Octal is less common, but the chmod command uses it when changing file permissions

How can we convert between bases?

- Binary → Hexadecimal
  - Take a binary value (0b10110110)
  - Group digits into sets of four (1011 and 0110)
    - 1011 = 1*2^3 + 0*2^2 + 1*2^1 + 1*2^0 = 11 = B
    - 0110 = 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0 = 6
  - 0b10110110 == 0xB6
- Hexadecimal → Binary
  - Convert each hexadecimal value to its 4-bit binary equivalent (0xE5)
  - E = 1110, 5 = 0101
  - 0xE5 == 0b11100101

- Can't remember the conversion? Take 0xB for instance, it is 11 in decimal (we check if each bits value would be set)

Each bits value	2^3 (8)	2^2 (4)	2^1 (2)	2^0 (1)
Is bit set?	Yes (11 – 8 = 3)	No (4 > 3)	Yes (3 – 2 = 1)	Yes (1 – 1 = 0)
Value	1	0	1	1

- So, we have 1011 which is equivalent to B

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

How can we convert between bases?

- Okay we don't really expect you to manually do the conversion, but it is a good thing to know!!

- During exams it would be encouraged to take advantage of python in the terminal to convert quickly and correctly between bases

  - type python3 in your terminal to open the python REPL
  - if you type a non decimal value (0b10010011 or 0x93) it will print the decimal equivalent

```
> python3
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 0b10010011
147
>>> 0x93
147
```

How can we convert between bases?

- Now to convert to binary, hexadecimal or octal we use these 3 functions
  - bin(0o223)
  - hex(0b10010011)
  - oct(147)

```
> python3
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> bin(0o223)
'0b10010011'
>>> hex(0b10010011)
'0x93'
>>> oct(147)
'0o223'
```

- And to close the python3 REPL you can press ctrl+d or type exit()

Bits and Bytes

- A bit is just a single binary *digit* just like how we have digits in decimal.
- A byte is a collection of 8 bits
  - This is a convenient size for computer because the size of types (char, int) generally fit into a multiple of bytes
- A register in MIPS stores 32-bit binary values (4 bytes)

Int types in C

- Sometimes we need more control over the size and type of integers we use. The library in C we use to have more integer types available is:
  - #include <stdint.h>
- u is prefixed if the integer is unsigned
- followed by int
- followed by size (8, 16, 32, 64) the number of bits
- followed by _t (naming convention for type)

- So, if we wanted an unsigned 16-bit integer type called bob in our program
  - uint16_t bob;

- Wait what does unsigned mean again? Simply unsigned numbers are strictly non-negative values (meaning you can store larger positive values)

Int types in C

- There are some substantial differences is the minimum and maximum values for some of these new types

type	Minimum	maximum
int8_t	-128	127
uint8_t	0	255
int16_t	-32768	32767
uint16_t	0	65535
int32_t	-2147483648	2147483647
uint32_t	0	4294967295
int64_t	-9223372036854775808	9223372036854775807
uint64_t	0	18446744073709551615

Example: q2

- How can we tell if an integer constant in a C program is decimal (base 10), hexadecimal (base 16), octal (base 8) or binary (base 2)?
  - decimal has no prefix – (int x = 15)
  - hexadecimal has 0x prefix – (int x = 0xF)
  - octal has 0 prefix – (int x = 017)
  - binary has 0b prefix – (int x = 0b1111)
- Is this good language design?
- Trivia: What base is the constant 0 in C?
  - According to the language grammar, technically 0 is octal
  - Though it doesn't really matter because 0 in any base is just 0

Example: q2

- Show what the following decimal values look like in 8-bit binary, 3-digit octal and 2-digit hexadecimal

decimal	binary	octal	hexadecimal
1			
8			
10			
15			
16			
100			
127			
200			

Example: q2

- Show what the following decimal values look like in 8-bit binary, 3-digit octal and 2-digit hexadecimal

decimal	binary	octal	hexadecimal
1	00000001	0001	0x01
8	00001000	0010	0x08
10	00001010	0012	0x0A
15	00001111	0017	0x0F
16	00010000	0020	0x10
100	01100100	0144	0x64
127	01111111	0177	0x7F
200	11001000	0310	0xC8

BITWISE OPERATIONS

Truth tables

- We have truth tables for logic AND/OR/NOT/XOR operations we may have seen before.

Logical AND

&	0	1
0		
1		

Logical OR

|	0	1
0		
1		

Logical NOT

~	0	1

Logical XOR

^	0	1
0		
1		

Truth tables

- We have truth tables for logic AND/OR/NOT/XOR operations we may have seen before.

Logical AND

&	0	1
0	0	0
1	0	1

Logical OR

|	0	1
0	0	1
1	1	1

Logical NOT

~	0	1
	1	0

Logical XOR

^	0	1
0	0	1
1	1	0

Bitwise

- A bitwise operation at its simplest level does the regular AND/OR/NOT/XOR (and a few others) on pairs of bits

- Consider we have two integers, and their binary representations are as follows:
  - 0b01101101
  - 0b10001001

Bitwise AND

```
  01101101
& 10001001
----------
  00001001
```

Bitwise OR

```
  01101101
| 10001001
----------
  11101101
```

Bitwise NOT

```
~ 01101101
----------
  10010010
```

Bitwise XOR

```
  01101101
^ 10001001
----------
  11100100
```

Bitwise Shift Operations

- We also have operations called a bitwise shifts, these will shift bits a specified number of times in a specified direction:
  - Left shift `<<`
  - Right shift `>>`
- Consider the binary value 0100:
  - 0100 << 1

    1000
  - 0100 >> 2

    0001
- Bits that are shifted "over the edge" are erased
- With an unsigned integer the new bits the enter from the opposite side will be 0's (uint8_t)
  - Sidenote: for signed integers this is not always the case and will be explored in week 7!

Bitwise Shift

- Bitwise shifts are an efficient way to multiple and divide unsigned integers by powers of two:
- Left Shift << (Multiplication)
  - Shifting left by n is equivalent to multiplying by 2^n
  - Consider the value 0011 (3 in decimal)
    - 0011 << 2 = 1100 (12)
    - $3 * 2^2 = 3 * 4 = 12$
- Right Shift >> (Division)
  - Shifting right by n is equivalent to dividing by 2^n
  - Consider the value 1100 (12 in decimal)
    - 1100 >> 1 = 0110 (6)
    - $12 / 2^1 = 12 / 2 = 6$

- Note: if we left shift too many times we can overflow and lose data if our result exceeds the integers maximum value
- Additionally, right shift division is integer division so 5 >> 1 results in 2 not 2.5

Bitwise in C

- The operators &, |, ~, ^, >> and << can be used just like any other arithmetic operators (+, -, *)
- However, it is important we always use these on unsigned types if possible (uint8_t etc)
  - As shifting unsigned types can cause some confusing bugs

```c
uint8_t x = 0xD2; // 0b11010010
uint8_t y = 0x4F; // 0b01001111
uint8_t res = 0;

// Bitwise AND
res = x & y;
// Bitwise OR
res = x | y;

// Bitwise AND/OR can be used similar to += -= also
res |= x;
res &= y;

// Bitwise NOT
res = ~x;
// Bitwise XOR
res = x ^ y;


// Bitwise Left Shift
res = x << 3;
// Bitwise Right Shift
res = x >> 3;
```

Bitwise in MIPS

- Similarly, we can also use these bitwise operations in MIPS between values in registers

```
#Bitwise AND
and     $t1, $t1, $t2

#Bitwise OR
or      $t1, $t1, $t2

#Bitwise NOT
not     $t1, $t1, $t2

#Bitwise XOR
xor     $t1, $t1, $t2

#Shift Right Logical
srl     $t1, $t1, 1

#Shift Left Logical
sll     $t1, $t1, 1
```

Example: q3

- Assume we have the following 16-bit values

```
uint16_t a = 0x5555; // 0101010101010101
uint16_t b = 0xAAAA; // 1010101010101010
uint16_t c = 0x0001; // 0000000000000001
```

- What are the values of the following expressions:

Expression	Result Binary	Result Hex
a | b		
a & b		
a ^ b		
a & ~b		
c << 6		
a >> 4		
a & (b << 1)		
b | c		

Example: q3

- Assume we have the following 16-bit values

```
uint16_t a = 0x5555; // 0101010101010101
uint16_t b = 0xAAAA; // 1010101010101010
uint16_t c = 0x0001; // 0000000000000001
```

- What are the values of the following expressions:

Expression	Result Binary	Result Hex
a | b	0b1111111111111111	0xFFFF
a & b	0b0000000000000000	0x0000
a ^ b	0b1111111111111111	0xFFFF
a & ~b	0b0101010101010101	0x5555
c << 6	0b0000000001000000	0x0040
a >> 4	0b0000010101010101	0x0555
a & (b << 1)	0b0101010101010100	0x5554
b | c	0b1010101010101011	0xAAAB

USING BITWISE

Why do we use bitwise?

- All of this doesn't seem massively useful yet, multiplication and division shifts could be, but most cases require / or *
- The main use of these operators is to read and write specific bit/s inside numbers and can allow us to store information in much more compactly

- Imagine we wanted 8 boolean values in an array for settings in our program
  - bool settings[8];
  - bool is typically 1 byte, that means we use 8 bytes to store these settings

- We could store all 8 Boolean values in a single byte using each individual bit as 1 for true 0 for false

uint8_t settings = 00010011

This would be the same as having settings[0], settings[1] and settings[4] as true (we go right to left)
Now we just need to learn how to write and extract this information

An example we may have seen

- You may have noticed that an instruction in MIPS is packed into a 32-bit integer to represent the instruction

✓	ADD	R_d, R_s, R_t	$R_d = R_s + R_t$	INTEGER OVERFLOW	000000ssssstttttddddd00000100000

000000sssssttttdddddd00000100000

- opcode (R-Type)
- source register 1
- source register 2
- destination register
- shamt (shift amount) only used in shift instructions
- funct (function code) tells the cpu the operation

Least and Most Significant bits

- We call the bits to the right least significant and the bits to the left most significant as it represents how impactful altering a bit in those positions are for a number
- Consider 74 in binary (0b01001010)
  - If we change the least significant bit (right-most) to 1
    - we have 0b01001011 which is 75 in decimal
  - If we change the most significant bit (left-most) to 1
    - we have 0b11001010 which is 202 in decimal

How do we set bits to 1

- Generally, we can use bitwise OR | to write 1 bits to a number
- If we have an unknown 8-bit binary number say ???????? and we wish to set the right most bit to 1
- We can take an 8-bit value where the least significant bit is 1, and the others 0 and use | with our value
  - 00000001 | ????????
  - = ???????1
  - This will ensure the least significant bit is 1 and the others are untouched
- Similarly, we could set the most significant bit using the same value and bit shifts
  - (00000001 << 7) | ????????
  - = 1???????

How do we set bits to 0

- We can use bitwise AND (&) to write bits to a number
- For instance, we have our mystery number ???????? and we wish to ensure the 3 least significant bits are 0

```
  ????????
& 11111000
----------
  ?????000
```

How do we read bits from a number?

- We can use bitwise AND (&) like writing 0 bits, however we just set unwanted bits to 0 for our result
- If we have our mystery number again ???????? and we want to read the 4 most significant bits (left-most) we can perform a similar operation

```
  ????????
& 11110000
----------
  ????0000
```

We effectively read the bits we want by setting all bits we are not interested in to 0

How do we read bits from a number?

```
  ????????
& 11110000
----------
  ????0000
```

- After reading these 4 bits, we may want to check what the value is equal to, however it is 16 times larger due to the 0's on the right
- We can use the bitwise right shift operator to move the 4 bits we care about the right-most position

????0000 >> 4 = 0000????

Example: q7.c

- From this week's lab exercises what does the main of sixteen_out do?

```c
long l = strtol(argv[arg], NULL, 0);
assert(l >= INT16_MIN && l <= INT16_MAX);
int16_t value = l;

char *bits = sixteen_out(value);
printf("%s\n", bits);

free(bits);
```

Example: q7.c

```
long l = strtol(argv[arg], NULL, 0);
assert(l >= INT16_MIN && l <= INT16_MAX);
int16_t value = l;

char *bits = sixteen_out(value);
printf("%s\n", bits);

free(bits);
```

- strlol converts a null-terminated string to an integer long
- The third parameter indicates the base to use (10 for decimal), a 0 means treat values starting with 0x as hexadecimal, 0 as octal and otherwise decimal
- the assert is to check if the value returned fits within 16 bits
- Lastly as the function returns an allocated string (malloc) we must free the memory once we are done

Example: q8.c

- Write a C function reverseBits(Word w), where word is an unsigned interger, which reverses all the bits in the variable w, for example if w = 0x01234567. We reverse it to change it as so.
  - 0000 0001 0010 0011 0100 0101 0110 0111
  - 1110 0110 1010 0010 1100 0100 1000 0000

Example: q8.c

- Write a C function reverseBits(Word w), where word is an unsigned interger, which reverses all the bits in the variable w, for example if w = 0x01234567. We reverse it to change it as so.
  - 0000 0001 0010 0011 0100 0101 0110 0111
  - 1110 0110 1010 0010 1100 0100 1000 0000

```
Word reverseBits(Word w) {
    Word ret = 0;
    for (unsigned int bit = 0; bit < 32; bit++) {
        Word wMask = 1u << (31 - bit);
        Word retMask = 1u << bit;
        if (w & wMask) {
            ret = ret | retMask;
        }
    }

    return ret;
}}
  return go(f, seed, [])
}
```

FIN